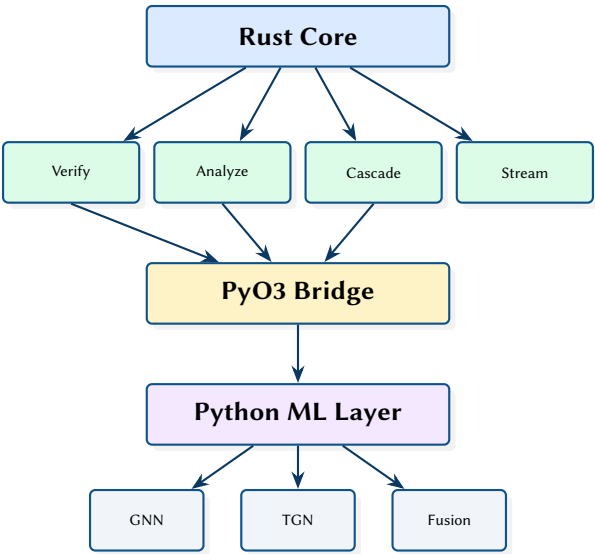


NetAssure: A GNN-Based Real-Time Network Analytics Platform

Formal Verification, Cascade Simulation, and Anomaly Detection

Simon Knight

March 2026 • Version 0.1.0



Abstract. This report presents NetAssure, a hybrid Rust/Python network analytics platform that provides six complementary analysis paradigms: formal verification via Header Space Analysis, graph-theoretic metrics, Monte Carlo failure cascade simulation, GNN-based anomaly detection, real-time streaming with WebSocket alerting, and topology optimisation. The Rust core (netassure-core, netassure-verify, netassure-analyze, netassure-cascade) handles performance-critical deterministic algorithms, while a Python layer provides PyTorch-based machine learning including temporal graph networks and multi-modal fusion scoring. PyO3 bindings bridge the two worlds, exposing verification, analysis, and cascade simulation to the Python ML pipeline.

Version	Date	Changes
0.1.0	March 2026	Initial architecture report

Contents

List of Design Decisions & Rules	2
1 Introduction and Motivation	3
2 System Architecture	3
2.1 End-to-End Dataflow	3
2.2 Crate Structure	3
2.3 Topology Model	4
2.4 REST API	4
2.5 Control Plane vs Data Plane Boundaries	4
3 Formal Verification	5
3.1 Header Space Propagation Model	5
4 Failure Cascade Simulation	5
4.1 Cascade Dynamics	5
4.2 Example Resilience Curve	6
5 Machine Learning Pipeline	7
5.1 Online Inference and Alerting	7
5.2 GNN Anomaly Detection	7
5.3 Temporal Graph Networks	7
5.4 Multi-Modal Fusion	7
6 Design Summary	8

List of Figures

1 End-to-end dataflow and execution boundaries across Rust and Python.	3
2 Trust boundary between deterministic analysis and probabilistic inference..	5
3 Conceptual header-space propagation and loop detection..	5
4 Round-based cascade progression used in the Monte Carlo simulator.	6
5 Illustrative resilience curve from Monte Carlo cascade simulation.....	6
6 Online inference and alerting pipeline.	7

List of Tables

1 NetAssure Rust crate organisation.	4
2 REST API endpoints.	4
3 Key design decisions and their rationale.	8

GNN	Graph Neural Network	3
TGN	Temporal Graph Network	7
HSA	Header Space Analysis	3
API	Application Programming Interface	4
ML	Machine Learning	3

List of Design Decisions & Rules

1	Design Rule: Hybrid Rust/Python Architecture	3
1	Bit-Set Header Representation	5
2	Fusion Architecture	8

1 Introduction and Motivation

Network operators need multiple analysis perspectives to understand complex topologies: formal verification alone cannot predict cascade failures, while machine learning alone cannot guarantee reachability invariants. NetAssure integrates six complementary paradigms into a single platform with a unified topology model.

The six paradigms are:

1. **Formal Verification** — Header Space Analysis (HSA), reachability proofs, loop detection, and traffic isolation checking.
2. **Graph Algorithms** — Centrality, community detection, path diversity, and robustness metrics.
3. **Failure Cascade Modelling** — Monte Carlo simulation with load redistribution and scenario comparison.
4. **Machine Learning (ML)/Graph Neural Network (GNN)** — Anomaly detection, temporal graph networks, and feature attribution via GNNExplainer.
5. **Real-Time Streaming** — Inference pipeline, anomaly detector, alert engine, and WebSocket streaming.
6. **Optimisation** — Critical node identification and redundancy recommendations.

: Hybrid Rust/Python Architecture

Deterministic algorithms (verification, graph metrics, cascade simulation) are implemented in Rust for performance and correctness. Machine learning workloads run in Python (PyTorch/PyTorch Geometric) where the ecosystem is mature. PyO3 bridges the two at the function call level, avoiding serialisation overhead.

2 System Architecture

2.1 End-to-End Dataflow

Figure 1 shows how topology data and telemetry flow through the system and where deterministic engines and ML inference sit.

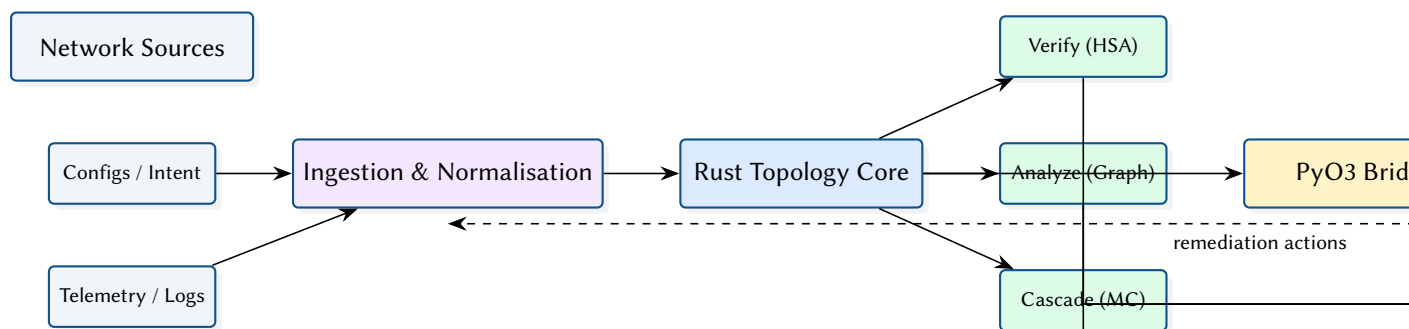


Figure 1. End-to-end dataflow and execution boundaries across Rust and Python.

2.2 Crate Structure

The Rust workspace contains four library crates and a CLI binary:

Table 1. NetAssure Rust crate organisation.

Crate	Role	Key Dependencies
netassure-core	Topology model, ingestion	petgraph, serde
netassure-verify	HSA , reachability	bit-set, petgraph
netassure-analyze	Graph algorithms	rustworkx-core
netassure-cascade	Monte Carlo simulation	rayon, rand
netassure-py	PyO3 bindings	pyo3
cli	Unified CLI	clap, axum, tokio

2.3 Topology Model

The core topology is a `petgraph::StableGraph` with typed node and edge attributes. Stable indices ensure that node/edge removal does not invalidate existing references—critical for cascade simulation where nodes are progressively disabled.

```
pub struct Topology {
    graph: StableGraph<NodeAttr, EdgeAttr>,
    node_index: HashMap<String, NodeIndex>,
}

pub struct NodeAttr {
    pub id: String,
    pub node_type: NodeType,
    pub capacity: f64,
    pub load: f64,
}
```

2.4 REST API

The ingestion system exposes an Axum-based HTTP Application Programming Interface ([API](#)) for real-time topology synchronisation and prediction queries:

Table 2. REST API endpoints.

Method	Path	Description
GET	/health	Liveness probe
GET	/health/ready	Readiness (topology synced?)
GET	/topology	Full topology JSON
GET	/predictions	GNN node predictions
GET	/anomalies	Recent anomaly history

2.5 Control Plane vs Data Plane Boundaries

NetAssure keeps a strict separation between what must be correct-by- construction (verification and deterministic analysis) and what is probabilistic (ML inference). [Figure 2](#) sketches the trust boundary used throughout the implementation.

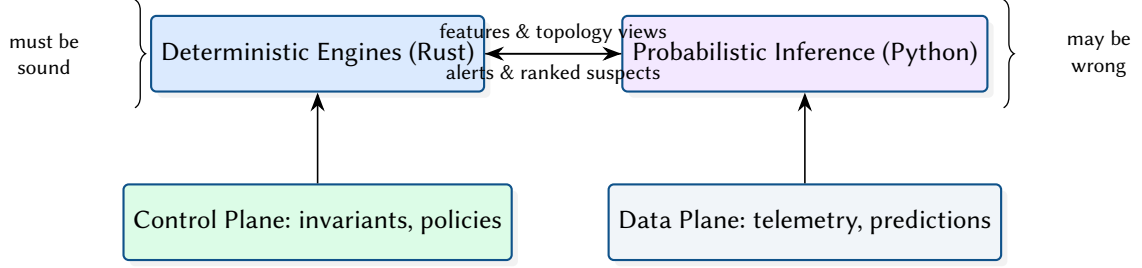


Figure 2. Trust boundary between deterministic analysis and probabilistic inference.

3 Formal Verification

3.1 Header Space Propagation Model

Figure 3 provides a conceptual diagram of header-space propagation across forwarding elements. This abstraction is used to derive reachability, isolation violations, and loop candidates.

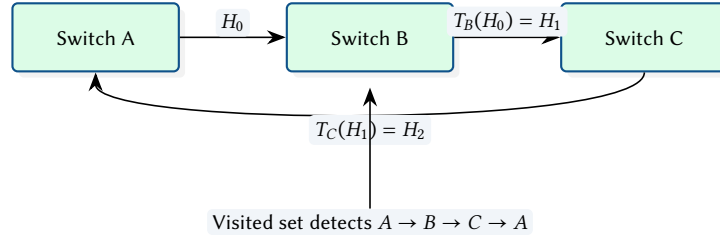


Figure 3. Conceptual header-space propagation and loop detection.

The verification engine implements HSA [1] over the topology graph. Each forwarding rule is modelled as a header-space transfer function; the engine computes reachability by composing transforms along all paths between source and destination.

Design Decision 1: Bit-Set Header Representation

Header spaces are represented as bit-sets rather than ternary vectors. This trades some expressiveness for $O(1)$ intersection and union operations, which dominate the verification runtime. For the network sizes targeted (hundreds of nodes, thousands of rules), this approach is sufficient and avoids the complexity of BDD-based representations.

Loop detection operates by tracking visited nodes during header-space propagation. If a packet's header matches a forwarding rule that would send it to an already-visited node, a loop is reported with the full cycle path.

4 Failure Cascade Simulation

4.1 Cascade Dynamics

Figure 4 illustrates the round-based cascade loop: initial failures remove capacity, load redistributes, and newly overloaded nodes fail until convergence.

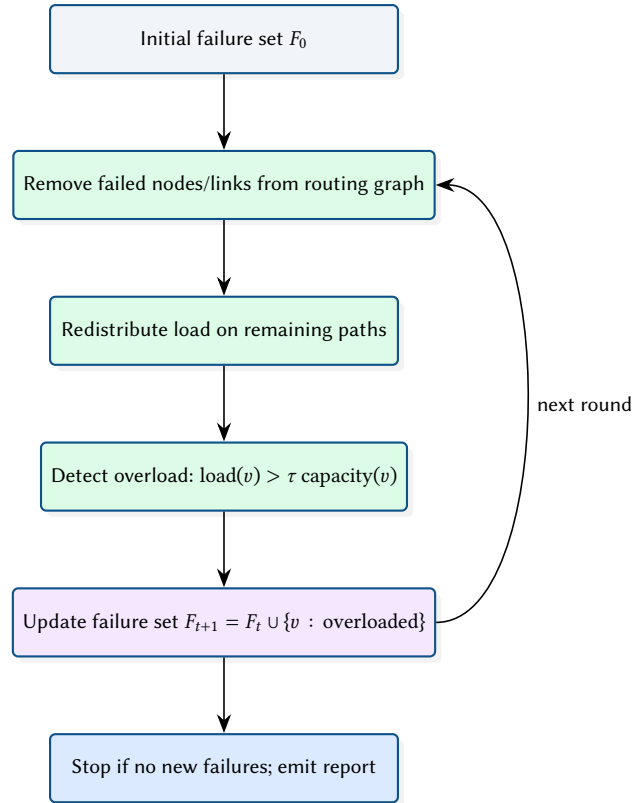


Figure 4. Round-based cascade progression used in the Monte Carlo simulator.

4.2 Example Resilience Curve

The Monte Carlo simulator produces distributions of cascade scope. Figure 5 shows an illustrative curve of median failed fraction vs. initial failure rate.

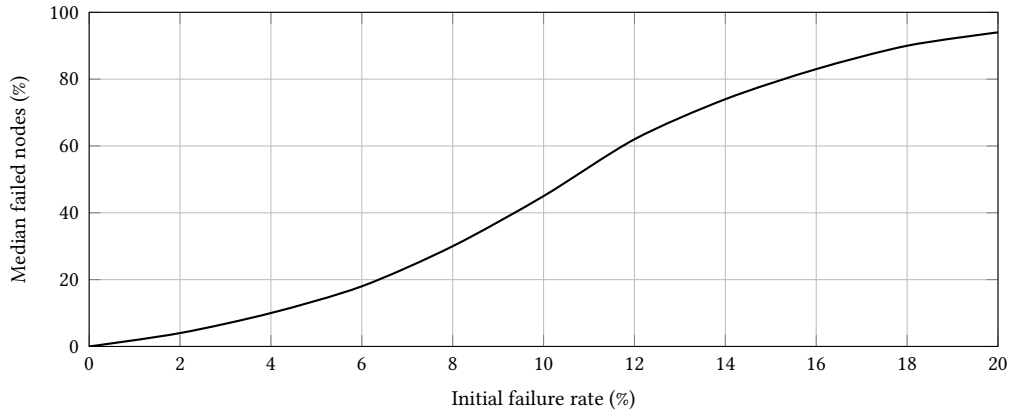


Figure 5. Illustrative resilience curve from Monte Carlo cascade simulation.

The cascade simulator models progressive network failure through load redistribution. When a node or link fails, its traffic is redistributed across remaining paths. If any node exceeds its capacity threshold, it fails in turn, potentially triggering a cascade.

```

pub fn simulate_cascade(
  topo: &mut Topology,
  initial_failures: &[NodeIndex],
  redistribution: RedistributionPolicy,
  capacity_threshold: f64,

```

```

) -> CascadeReport {
  let mut round = 0;
  let mut failed: HashSet<NodeIndex> = initial_failures.iter().cloned().collect();
  loop {
    let overloaded = find_overloaded(&topo, &failed, capacity_threshold);
    if overloaded.is_empty() { break; }
    failed.extend(overloaded);
    redistribute_load(topo, &failed, &redistribution);
    round += 1;
  }
  CascadeReport { rounds: round, total_failed: failed.len(), .. }
}

```

Monte Carlo mode runs thousands of iterations with random initial failure sets (parameterised by failure percentage), using Rayon for parallel execution. Each iteration produces a cascade depth and scope, enabling statistical analysis of network resilience.

5 Machine Learning Pipeline

5.1 Online Inference and Alerting

Figure 6 sketches the online loop used by the real-time detector: ingest events, update features, run inference, and push alerts to operators.

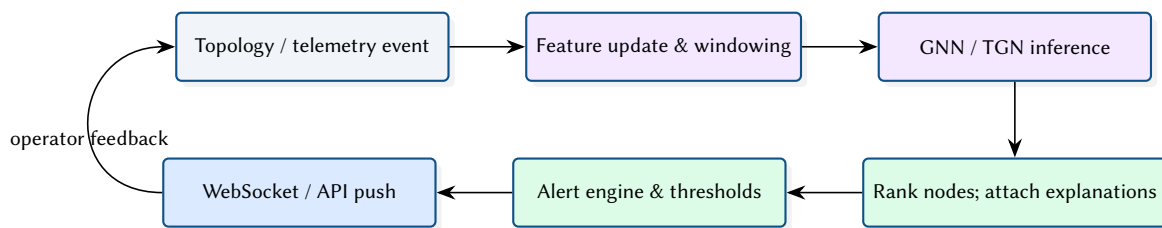


Figure 6. Online inference and alerting pipeline.

Insight:
Scenario comparison mode runs two cascade experiments with different topologies (e.g. before and after adding redundancy) and produces a diff report highlighting improvements in cascade depth, scope, and critical node rankings.

5.2 GNN Anomaly Detection

The GNN model [2] operates on a 14-dimensional feature schema per node (degree centrality, betweenness, load ratio, interface counts, etc.) and produces per-node anomaly scores.

5.3 Temporal Graph Networks

The Temporal Graph Network (TGN) model processes time-series topology events (link flaps, load changes) to predict future anomalies. It maintains a per-node memory vector updated at each event, enabling the model to capture temporal patterns such as recurring failures or gradual degradation.

5.4 Multi-Modal Fusion

The fusion scorer combines three signal types:

- **Topology embeddings** (14-dim feature schema)
- **Metrics sequence** (5-dim: latency, loss, jitter, throughput, error rate)
- **Log embeddings** (768-dim from a pre-trained encoder)

Design Decision 2: Fusion Architecture

The fusion model concatenates the three embedding vectors after per-modality projection heads, then passes through a two-layer MLP with dropout. This late-fusion approach allows each modality to be independently pre-trained and evaluated, simplifying debugging when one signal source is noisy or unavailable.

6 Design Summary

Table 3. Key design decisions and their rationale.

Decision	Rationale
Hybrid Rust/Python	Deterministic algorithms in Rust for speed; ML in Python for ecosystem maturity
PyO3 bindings	Zero-copy where possible; avoids JSON serialisation on the hot path
StableGraph	Node removal during cascade simulation must not invalidate existing indices
Bit-set HSA	$O(1)$ set operations; sufficient for target network sizes
Late fusion scoring	Independent modality training; graceful degradation when a signal is missing

References

- [1] P. Kazemian, G. Varghese, and N. McKeown, “Header space analysis: Static checking for networks,” *NSDI*, 2012.
- [2] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *ICLR*, 2017.